Web Scrapper

n8n Automation Workflow — Technical README & Build Guide

Workflow Name	Web Scapper (Web Scrapper)
Platform	n8n (self-hosted / cloud)
Trigger Type	Form Submission (Webhook-based Form Trigger)
Primary Function	Scrape a webpage's links & images, log to Google Sheets
Destination	Google Sheets — "WEB SCRAPPER" spreadsheet, "Scrapped Data" tab
Node Count	9 nodes
Status in file	Active ("active": true)

1. Overview

This n8n workflow is a lightweight **web scraping automation**. A user submits a URL through a built-in n8n form, the workflow fetches that page's raw HTML, extracts every hyperlink (`<a href>`) and every image source (`<img src>`) on the page, normalizes/deduplicates them, converts them into Google Sheets formulas, and appends the results as rows in a connected Google Sheet.

The workflow contains a hardcoded base-URL assumption for **Pinterest** (`https://www.pinterest.com`) used to resolve relative links, which means in its current form it is tuned for scraping Pinterest-style pages, though the HTTP fetch and HTML extraction logic works generically on any URL.

What it is useful for

- Quickly collecting every outbound link and image URL from a single web page.
- Building a running spreadsheet log of scraped links/images for later review (e.g. content curation, competitor research, mood-boarding from Pinterest).
- A reusable base template that can be adapted to other sites by changing the base-URL logic.

2. Workflow Architecture

The 9 nodes execute strictly in sequence — there is no branching or conditional logic in this workflow. The high-level data flow is:

```
1. On form submission → captures the URL typed into the form
2. Code in JavaScript → pulls the URL field out of the form payload
3. HTTP Request → fetches the raw HTML of that URL
4. HTML → extracts all <a href> links and <img src> images from the HTML
5. Code in JavaScript1 → resolves relative links to absolute Pinterest URLs + de-duplicates
6. Code in JavaScript2 → flattens links/images arrays into individual typed items
7. Code in JavaScript4 → re-splits items back into two arrays and zips them into link/image row
pairs
8. Code in JavaScript3 → wraps each row in Google Sheets HYPERLINK() and IMAGE() formulas
9. Append or update row in sheet → writes/updates rows in the Google Sheet
```

Figure 1 — Linear execution order derived from the workflow's node connections graph.

3. Node-by-Node Breakdown

Detailed purpose, node type, and key configuration for every node in the workflow, in execution order.

1	**On form submission**	**Form Trigger (n8n-nodes-base.formTrigger)**

Purpose: Entry point of the workflow. Renders a public n8n form titled "Web Scrapper" with a single text field, `URL`. Submitting the form starts the workflow run and generates a webhook.

Configuration:
• Form Title: "Web Scrapper"
• Field: URL (single text input)
• typeVersion: 2.5

2	**Code in JavaScript**	**Code (n8n-nodes-base.code)**

Purpose: Reads the submitted form data and extracts just the `URL` value, logging it to the console for debugging, then passes it downstream as a clean `{ url }` object.

Configuration:
• alwaysOutputData: true (ensures a run even if input is empty)
• retryOnFail: false

Logic (code):

```
const items = $input.all();
const url = items[0].json.URL;
console.log(url);
return [ { json: { url } } ];
```

3	**HTTP Request**	**HTTP Request (n8n-nodes-base.httpRequest)**

Purpose: Performs a GET request to the submitted URL (`={{$json.url}}`) and retrieves the page's raw HTML response body.

Configuration:
• URL: expression bound to {{$json.url}} from the previous node
• Options: none customized (default method GET)

4	**HTML**	**HTML Extract (n8n-nodes-base.html, operation: extractHtmlContent)**

Purpose: Parses the fetched HTML and extracts two arrays using CSS selectors: all hyperlink targets and all image sources.

Configuration:
• Extraction 1 — key: links | selector: a | attribute: href | returns an array
• Extraction 2 — key: images | selector: img | attribute: src | returns an array

5	Code in JavaScript1	Code (n8n-nodes-base.code)

Purpose: Cleans up the raw extracted links: converts relative paths into absolute URLs using a hardcoded Pinterest base URL, then removes duplicate links and duplicate images.

Logic (code):

```
const baseUrl = "https://www.pinterest.com";

const links = ($json.links || []).map(link => {
if (!link) return null;
if (link.startsWith("http")) return link;
if (link.startsWith("/")) return baseUrl + link;
return baseUrl + "/" + link;
}).filter(Boolean);

const images = [...new Set($json.images || [])];

return [{ json: { links: [...new Set(links)], images } }];
```

Note: The base URL is hardcoded to Pinterest. Scraping any other domain will still work for absolute links, but relative links will be incorrectly resolved against pinterest.com.

6	Code in JavaScript2	Code (n8n-nodes-base.code)

Purpose: Flattens the two arrays (links, images) that live inside one item into many individual items, each tagged with a `type` field of either "link" or "image". This converts a 1-item-with-arrays structure into an n8n-friendly multi-item list.

Logic (code):

```
const links = $json.links || [];
const images = $json.images || [];

return [
...links.map(link => ({ json: { type: "link", value: link } })),
...images.map(image => ({ json: { type: "image", value: image } }))
];
```

7	Code in JavaScript4	Code (n8n-nodes-base.code)

Purpose: Re-separates the flattened items back into two lists (links vs images) and re-pairs them index-by-index into row objects, so each output item represents one spreadsheet row containing a link and an image (padding with empty strings where one list is longer than the other).

Logic (code):

```
const items = $input.all();
const links = items.filter(i => i.json.type === "link").map(i => i.json.value);
const images = items.filter(i => i.json.type === "image").map(i => i.json.value);

const maxLen = Math.max(links.length, images.length);
const rows = [];
for (let i = 0; i < maxLen; i++) {
rows.push({ json: { link: links[i] || "", image: images[i] || "" } });
}
return rows;
```

Note: Because links and images are paired purely by array index (not by which link actually contains which image), row pairings are positional, not semantically linked.

8	Code in JavaScript3	Code (n8n-nodes-base.code)

Purpose: Wraps each row's plain URL strings in Google Sheets formula syntax so that, once written to the sheet, links render as clickable hyperlinks and images render as inline thumbnails.

Logic (code):

```
const items = $input.all();
return items.map(item => ({
json: {
link: `=HYPERLINK("${item.json.link}", "${item.json.link}")`,
image: `=IMAGE("${item.json.image}")`
}
}));
```

9	Append or update row in sheet	Google Sheets (n8n-nodes-base.googleSheets, operation: appendOrUpdate)

Purpose: Writes the final formula-wrapped rows into the connected Google Sheet, appending new rows or updating existing ones when a matching "Links" value is already present.

Configuration:

- Spreadsheet: "WEB SCRAPPER"
- Sheet/tab: "Scrapped Data" (gid=0)
- Column mapping: Links = {{$json.link}}, Images = {{$json.image}}
- Matching column: Links (used to decide append vs. update)
- Credential: Google Sheets OAuth2 ("Google Sheets account")

4. Example Data Flow

A concrete walk-through of how one form submission moves through the workflow:

Stage	Example Value
Form input	URL = https://www.pinterest.com/search/pins/?q=travel
After Code in JavaScript	{ url: "https://www.pinterest.com/search/pins/?q=travel" }
After HTTP Request	Raw HTML string of the page
After HTML node	{ links: ["/pin/123", "https://ext.com/a"], images: ["/img1.jpg", "/img1.jpg"] }
After Code in JavaScript1	{ links: ["https://www.pinterest.com/pin/123", "https://ext.com/a"], images: ["/img1.jpg"] }
After Code in JavaScript2	2 items: {type: link, value: ...}, {type: link, value: ...}, {type: image, value: ...}
After Code in JavaScript4	{ link: "https://www.pinterest.com/pin/123", image: "/img1.jpg" } (+ 1 more row)
After Code in JavaScript3	{ link: '=HYPERLINK("...", "...")', image: '=IMAGE("/img1.jpg")' }
Final sheet row	Links column = clickable link | Images column = rendered thumbnail

5. Step-by-Step Build Guide (Recreating this Workflow from Scratch)

Follow these steps in order inside the n8n editor to rebuild the workflow node by node.

Step 1 — Create the trigger	Add a **Form Trigger** node. Set the form title to "Web Scrapper" and add one field labeled `URL`. This generates a public form URL / webhook that starts the workflow.
Step 2 — Extract the submitted URL	Add a **Code** node connected to the trigger. Use JavaScript to read `$input.all()[0].json.URL` and return it as `{ json: { url } }`.
Step 3 — Fetch the page	Add an **HTTP Request** node. Set the URL field to the expression `{{$json.url}}` so it dynamically fetches whatever page was submitted.
Step 4 — Extract links and images	Add an **HTML** node with operation `Extract HTML Content`. Define two extraction values: one for `a` tags returning the `href` attribute as an array (key: links), and one for `img` tags returning the `src` attribute as an array (key: images).
Step 5 — Normalize and de-duplicate	Add a **Code** node. Write logic to prefix relative links with your target site's base URL (e.g. Pinterest), and de-duplicate both the links and images arrays using `Set`.
Step 6 — Flatten into individual items	Add a **Code** node that maps the links array and images array into separate n8n items, each carrying a `type` field ("link" or "image") and a `value` field.

Step 7 — Re-pair links with images	Add a **Code** node that filters items back into two arrays by type, then zips them together by index into row objects of the shape `{ link, image }`.
Step 8 — Wrap as Sheets formulas	Add a **Code** node that wraps each row's link in a `HYPERLINK()` formula string and each image URL in an `IMAGE()` formula string, so Google Sheets renders them as clickable links and inline thumbnails.
Step 9 — Write to Google Sheets	Add a **Google Sheets** node with operation `Append or Update Row`. Connect your Google Sheets OAuth2 credential, select the target spreadsheet and tab, map the "Links" and "Images" columns to the incoming `link` / `image` fields, and set "Links" as the matching column.
Step 10 — Wire the connections and activate	Connect the nodes in the exact order above (trigger → code → HTTP → HTML → code1 → code2 → code4 → code3 → Google Sheets), save, and toggle the workflow to **Active** so the form endpoint stays live.

6. Prerequisites & Setup

Credentials required

- **Google Sheets OAuth2** credential connected in n8n, with edit access to the destination spreadsheet.

Target spreadsheet structure

A Google Sheet named **"WEB SCRAPPER"** containing a tab named **"Scrapped Data"** with at minimum two columns: `Links` and `Images`.

n8n environment

- Any self-hosted or cloud n8n instance supporting Form Trigger v2.5, HTTP Request v4.4, HTML v1.2, Code v2, and Google Sheets v4.7 (or newer compatible versions).
- Outbound internet access from the n8n instance to reach arbitrary target URLs and the Google Sheets API.

7. Configuring the Google Sheets OAuth2 Credential

The final node in this workflow ("Append or update row in sheet") needs a working Google Sheets OAuth2 credential before it can write anything. There are two ways to set this up in n8n: the quick built-in option, and the manual Google Cloud Console option (needed for self-hosted instances or production use). Both are covered below.

Option A — Quick setup (n8n Cloud, using n8n's built-in Google OAuth app)

1. In your n8n workflow, open the "Append or update row in sheet" node.
2. Under **Credential to connect with**, click the dropdown and select **Create New Credential**.
3. Choose **Google Sheets OAuth2 API** as the credential type.
4. Click **Sign in with Google** / **Connect my account**. This opens a Google consent screen using n8n's shared OAuth app.
5. Log in with the Google account that owns (or has edit access to) the target spreadsheet, and approve the requested Sheets/Drive scopes.
6. n8n will automatically store the access + refresh token as a reusable credential named "Google Sheets account" (or whatever name you give it).
7. Select that credential in the node, then pick the **WEB SCRAPPER** spreadsheet and **Scrapped Data** sheet from the resource dropdowns.

Option A only works on n8n Cloud or instances where the default Google OAuth client is enabled; it is the fastest path but relies on n8n's shared app being approved for your account/organization.

Option B — Manual setup with your own Google Cloud OAuth client (self-hosted / production)

1. Go to the **Google Cloud Console** (console.cloud.google.com) and create a new project (or select an existing one).
2. Open **APIs & Services** → **Library**, search for **Google Sheets API**, and click **Enable**. Also enable the **Google Drive API** (needed for file/spreadsheet lookup).
3. Open **APIs & Services** → **OAuth consent screen**. Choose **External** (or **Internal** if using Google Workspace), fill in the app name, support email, and developer contact, then save.
4. Under **Scopes**, add the Sheets and Drive scopes (e.g. `.../auth/spreadsheets` and `.../auth/drive.file`).
5. Under **Test users** (if the app is in Testing mode), add the Google account email that will own the credential.
6. Go to **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth client ID**. Choose application type **Web application**.
7. In n8n, open the Google Sheets node → Credential dropdown → **Create New Credential** → **Google Sheets OAuth2 API**, and copy the **OAuth Redirect URL** n8n displays.
8. Paste that redirect URL into the **Authorized redirect URIs** field of the OAuth client you just created in Google Cloud Console, then save.
9. Back in Google Cloud Console, copy the generated **Client ID** and **Client Secret**.
10. Paste the Client ID and Client Secret into the matching fields on the n8n credential form, then click **Sign in with Google** / **Connect** and approve access.

11. Once connected, n8n shows the credential status as **Connected**. Save the credential and select it in the "Append or update row in sheet" node.

Sharing the spreadsheet

- Make sure the Google account used for the credential has at least **Editor** access to the "WEB SCRAPPER" spreadsheet (open the sheet → Share → add that account, or use the same account that owns the sheet).
- If the spreadsheet lives in a Shared Drive or a different Google Workspace domain, confirm the OAuth app's scopes and domain-wide access allow the credential's account to reach it.

Verifying the credential

- In the Google Sheets node, use the **spreadsheet** dropdown — if the credential is valid it should list "WEB SCRAPPER" (and any other accessible sheets) without an authentication error.
- Run the node once manually ("Execute step") with a test row to confirm it writes to the "Scrapped Data" tab successfully before activating the whole workflow.

8. Known Limitations

- **Hardcoded base URL:** relative links are always resolved against `https://www.pinterest.com`, so scraping non-Pinterest sites will mis-resolve any relative hrefs.
- **Positional pairing:** the link ↔ image pairing in the final rows is based on array index, not on which image actually belongs to which link, so row pairings can be semantically meaningless.
- **No pagination or JS rendering:** the HTTP Request node fetches static HTML only; content rendered client-side via JavaScript (common on Pinterest) will not be captured.
- **No error handling:** there is no branch for failed HTTP requests, empty results, or invalid URLs.
- **No de-duplication against existing sheet rows:** de-duplication only happens within a single run, not against rows already present in the sheet (aside from the Links match/update key).

9. Suggested Improvements

- Replace the hardcoded Pinterest base URL with a dynamically derived origin from the submitted URL.
- Add an **IF** / error-handling branch after the HTTP Request node to catch failed fetches.
- Pair images to links contextually (e.g. by parsing the DOM structure) instead of by array index.
- Add rate-limiting or a wait node if scraping multiple pages in sequence.
- Consider a headless-browser fetch (e.g. Puppeteer/Playwright node) for JavaScript-rendered pages.

Generated documentation based on analysis of the exported workflow file `Web_Scapper.json`.